

Second Lab Assignment: Cache Simulator - Report

Francisco Fonseca - 102492; Gonalo Rua - 102604; Joo Gouveia - 102611

4.1 Directly-Mapped L1 Cache

For the first task we started by changing the way we calculate the offset (6 bits), index (8 bits) and tag (18 bits). We also changed the structure Cache so that we have multiple lines (array of size L1_BLOCK_COUNT). To access the right location inside the L1Cache array, we calculated the physical address of the corresponding block, multiplying the index with the block size. Finally, to access DRAM in case of miss with dirty flag, we had to calculate the physical address of the previous tag in that block on the DRAM array. The rest of the code remained practically unchanged.

```
void accessL1(uint32_t address, uint8_t *data, uint32_t mode) {
    [...]
    if (SimpleCache.init == 0) {
        for (int i = 0; i < L1_BLOCK_COUNT; i++)
            SimpleCache.line[i].Valid = 0;
        SimpleCache.init = 1;
    }
    /* get offset (word + byte), word offset, index and tag */
    offset = address & 0x0000003F; word_offset = offset >> 2;
    index = (address >> 6) & 0x000000FF; tag = address >> 14;
    CacheLine *Line = &SimpleCache.line[index]; // Cache Line structure
    /* get block address in DRAM */
    mem_address = address >> 6;
    mem_address = mem_address << 6;
    /* block and word address inside the cache */
    cache_block_address = (index * BLOCK_SIZE);
    cache_address = (index * BLOCK_SIZE) + (word_offset * WORD_SIZE);

    if (!Line->Valid || Line->Tag != tag) { // if block not present - miss
        [...]
        if ((Line->Valid) && (Line->Dirty)) { // line has dirty block
            /* temp_address is DRAM address for the dirty tag */
            temp_address = index << 6;
            temp_address |= Line->Tag << 14;
            accessDRAM(temp_address, L1Cache + cache_block_address, MODE_WRITE);
        }
        memcpy(L1Cache + cache_block_address, temp_block, BLOCK_SIZE);
        [...]
    }
    if (mode == MODE_READ) { // read data from cache line
        memcpy(data, L1Cache + cache_address, WORD_SIZE);
        time += L1_READ_TIME;
    } else if (mode == MODE_WRITE) { // write data from cache line
        memcpy(L1Cache + cache_address, data, WORD_SIZE);
        time += L1_WRITE_TIME;
        Line->Dirty = 1;
    }
    [...]
}
```

4.2 Directly-Mapped L2 Cache

To accomplish the second task we reused the code of the Directly-Mapped L1 Cache, since the logic of the Directly-Mapped L2 Cache is almost the same, just changing the size of the index (to 9 bits since the cache size is double the size). Also we had to change the L1 Cache code for when a miss occurs it reads from the L2 Cache and when there is a miss with dirty flag we write to the L2 Cache.

```
void accessL1(uint32_t address, uint8_t *data, uint32_t mode) {
    [...]
    if (!Line->Valid || Line->Tag != tag) { // if block not present - miss
        accessL2(mem_address, temp_block, MODE_READ); // try L2
    }
```

```

        if ((Line->Valid) && (Line->Dirty)) { // line has dirty block
            [...]
            accessL2(temp_address, L1Cache + cache_block_address, MODE_WRITE);
        }
    [...]
void accessL2(uint32_t address, uint8_t *data, uint32_t mode) {
    [...]
    if (L2Cache_status.init == 0) {
        for (int i = 0; i < L2_BLOCK_COUNT; i++)
            L2Cache_status.linesL2[i].Valid = 0;
    [...]
    index = (address >> 6) & 0x000001FF; tag = address >> 15;
    CacheLine *Line = &L2Cache_status.linesL2[index]; // Cache Line structure
    [...]
    if (!Line->Valid || Line->Tag != tag) { // if block not present - miss
        [...]
        if ((Line->Valid) && (Line->Dirty)) { // line has dirty block
            temp_address = index << 6;
            temp_address |= Line->Tag << 15;
            accessDRAM(temp_address, L2Cache + cache_block_address, MODE_WRITE);
        }
    [...]
}

```

4.3 2-Way L2 Cache

To add 2-way associativity to the L2 cach we only had to change the L2 cache code, while the first level cache remained unchanged. We created the struct CacheSet, which contains one array of type CacheLine. The L2 cache struct now stores one array of type CacheSet instead of CachLine. When we intend to load a block to a full set, we use the LRU policy to decide which blocks is replaced.

```

void accessL2(uint32_t address, uint8_t *data, uint32_t mode) {
    [...]
    if (L2Cache_status.init == 0) {
        for (int i = 0; i < L2_SET_COUNT; i++) {
            for (int j = 0; j < L2_ASSOCIATIVITY; j++)
                L2Cache_status.setsL2[i].line[j].Valid = 0;
        }
    [...]
    index = (address >> 6) & 0x000000FF; tag = address >> 14;
    CacheSet *Set = &L2Cache_status.setsL2[index]; // Cache Line structure
    [...]
    cache_set_address = index * BLOCK_SIZE * L2_ASSOCIATIVITY;
    /* Checking if the desired block is loaded*/
    int line = -1;
    for (int i = 0; i < L2_ASSOCIATIVITY; i++)
        if (Set->line[i].Valid && Set->line[i].Tag == tag)
            line = i;
    /* Didnt find the block*/
    if (line == -1) {
        [...]
        /* Trying to find an empty slot*/
        for (int i = 0; i < L2_ASSOCIATIVITY; i++)
            if (!Set->line[i].Valid)
                line = i;
        if (line == -1)
            line = Set->Lru;
        /* Decided which line to replace inside the set*/
        Line = &Set->line[line];
        cache_block_address = cache_set_address + line * BLOCK_SIZE;
        [...]
    } else {
        Line = &Set->line[line];
        cache_block_address = cache_set_address + line * BLOCK_SIZE;
    [...]
}

```